CHAPTER 1

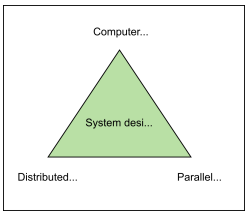
INTRODUCTION

1 Introduction to Modern System Design 2
2 Why Every Developer Should Learn System Design 4
3 Course Structure for Modern System Design 8

Get familiar with System Design and the course structure, including prerequisites that set the foundation for mastering System Design Interviews.

1 Introduction to Modern System Design

1.1 Understanding System Design



System design is the process of defining components and their integration, APIs, and data models to build large-scale systems that meet a specified set of functional and non-functional requirements.
System design uses the concepts of computer networking, parallel computing, and distributed systems to craft systems that scale well and are performant. Distributed systems scale well by nature. However, distributed systems are inherently complex. The discipline of system design helps us tame this complexity and get the work done.
System design aims to build systems that are reliable, effective, and maintainable, among other characteristics.

- **Reliable systems** handle faults, failures, and errors.
- **Effective systems** meet all user needs and business requirements.
- **Maintainable systems** are flexible and easy to scale up or down. The ability to add new features also comes under the umbrella of maintainability.

1.2 Modern System Design using building blocks

We have separated out commonly used design elements, such as load balancers, as the basic building blocks for high-level system design. This serves two purposes. First , it allows us to discuss all the building blocks in detail and discuss their interesting mini-design problems. Second , when we tackle a design problem, we can concentrate on problem-specific aspects, mention the building block we'll use, and how we'll use it. This helps us remove duplicate discussions of commonly occurring design elements.
We have identified sixteen building blocks that are crucial in designing modern systems.

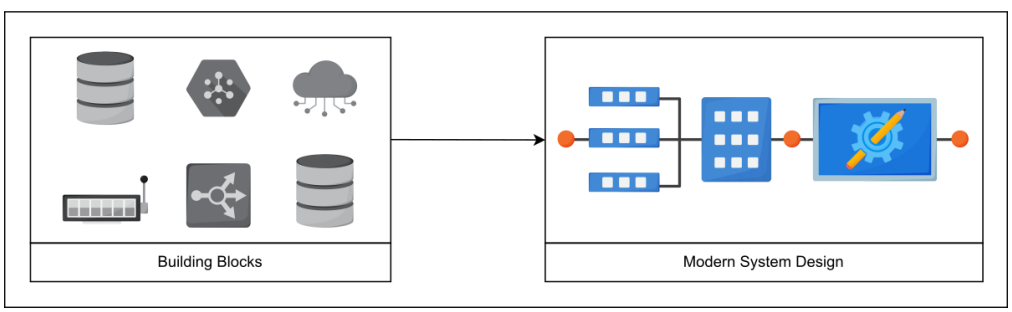


Figure 1.1: Modern systems are designed using fundamental building blocks

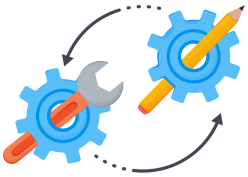
1.3 About this System Design course

This course is about designing systems that scale with increasing users, remain available even under different faults, and meet functional goals with good performance. Real -world system building is an iterative process where we start with a reasonably good design, measure how it performs, and improve the design in the next iteration.
The focus of this course is to immerse ourselves into carefully-selected system design endeavors to enable ourselves to tackle any novel design problem, be it in a systems design interview or a task at the office. This course aims to teach concepts instead of giving out boilerplate designs. Some gaps that this course aims to fill are listed below.

A fresh look at system design: Many system design courses provide a formula to attack a specific problem. This might seem attractive in a high-stress situation like an interview, but it might encourage memorizing a design solution instead of actually understanding the problem and devising an appropriate solution. If system design were that formulaic, then we probably wouldn't need people for system designing. System design is as much an art as it is a science, and attacking a design problem from the first principles gives a fresh feel to it.

Going deep and broad: We tackle some traditional problems, but with added in-depth discussions on them. We give proper rationale for why we use some components despite their tradeoffs. For example, we explain why we use a particular database, a caching system, or a load balancing technique in a design.
We address some new design problems as well that touch upon not only scalability but also availability, maintainability, consistency, and fault-tolerance. Collectively , traditional and new problems cover all aspects of modern system design activity. Our hope is that this course prepares learners to effectively tackle any new design problem they encounter.
Real systems are complex and, often, we might need to make appropriate assumptions to properly scope a problem. We cover problems in more detail to properly grasp the real-world systems.

Iterative process: Systems, in reality, improve over iterations. We often start with something simple, but when bottlenecks arise in one or more of the system's parts, a new design becomes necessary. In some design problems, we make one design, identify bottlenecks, and improve on it. Working under time constraints might not permit iterations on the design. However, we still recommend two iterations—first, where we do our best to come up with a design (that takes about 80 percent of our time), and a second iteration for improvements. Another choice is to change things as we figure out new insights. Inevitably, we discover new details as we spend more time working with a problem.



Interactive learning: We provide ample opportunities to get experience with system design. Some design problems guide learners through many steps to design a system. We also have a few examples where the learner designs the full system end-to-end without any guided steps. We reinforce the important concepts by testing learners with questions and quizzes.

1.4 Who should take this System Design course?

System design is for any software engineer who wants to advance in their career.

- **Interview preparation:** Lately, system design is becoming an important part of software development interviews. This course helps software engineers prepare for interviews. We have an elaborate guide on preparation for a system design interview in this course for learners who are interested.

- **Software developers:** System design is primarily for back-end developers who aim to become principal engineers or solution architects. It's because these engineers handle actual user data. Once the data is submitted to back-end systems by the frontend, effective handling of data makes the overall application successful. Having said that, full stack or front-end developers may also want to learn system design to improve their work. At the same time, **support engineers** (also called **SRE** s) who work on-call in the production environment have to deal with all sorts of problems daily. Therefore, system design concepts enable SREs to efficiently find the root causes of complex problems.

- **Product/project/engineering managers:** A big challenge in product/project/engineering management is to build systems that scale well and perform effectively over time. Managers that are aware of system design can steer the design of large-scale performant systems. Therefore, it is imperative for managers to understand system design concepts to lead the design and development of successful applications.

- **System design learners:** System design is an interesting subject, and people in tech domains can greatly benefit from learning system design. This course helps a learner understand how giant tech companies design and build successful applications from scratch and improve them over time. Other learners may want to develop their idea into a large application by learning from this course.

1.5 Prerequisites for this System Design course

We assume that you know the fundamental concepts of a distributed system.
System design borrows many great concepts from distributed systems. We have an excellent course for learners who are interested in revisiting and reviewing their concepts on distributed systems. Apart from distributed systems, some basic concepts on computer networking and operating systems are also helpful before taking this course.

2 Why Every Developer Should Learn System Design

Modern software increasingly depends on distributed systems, interconnected components, and high user expectations. Developers across all roles benefit from understanding how these systems work, enabling them to build applications that perform reliably at scale.
This lesson focuses on the motivations for learning System Design, why it matters, why it's now indispensable, and why every developer should embrace it early in their career.

2.1 Why do we need System Design?

Software development has evolved from simple, standalone applications to complex, distributed ecosystems spanning mobile, web, and cloud environments. What once was the domain of large tech companies is now the baseline for nearly all modern systems.
Note: System Design bridges the gap between writing features and engineering systems that withstand growth, failure, and real-world demands. It gives developers the tools to manage complexity, make informed trade-offs, and ensure their applications remain fast, reliable, and resilient, even under unpredictable conditions.

Figure 1.2: An overview of interconnected components

By mastering System Design, developers gain the ability to reason about architecture, choose appropriate patterns, anticipate bottlenecks, and design systems that meet core objectives, such as performance, availability, and user experience.

2.2 Why every developer should learn System Design

All developers interact with distributed systems, whether directly or through APIs, services, databases, or cloud infrastructure. System Design knowledge improves how developers build and integrate these systems, regardless of the domain. Here 's how it empowers different types of developers and teams:

- **Mobile developers:** Connect to services for authentication, messaging, search, payments, and storage. System Design helps them coordinate these components efficiently and build responsive, fault-tolerant apps.
- **Backend developers:** Make daily decisions about concurrency, databases, caching, scaling, and failure handling. System Design enhances its ability to design robust services and balance trade-offs, such as latency vs. consistency.
- **Game developers:** Build real-time multiplayer systems that demand low latency and state synchronization. Understanding distributed communication, replication, and load handling is essential.
- **Frontend developers:** Consume distributed systems via APIs. System Design knowledge helps them optimize API usage, reduce latency, and avoid cascading errors caused by backend interactions.
- **Data engineers:** Manage pipelines, streaming systems, and batch workflows. System Design helps them build reliable, scalable data platforms resistant to failures and skewed workloads.
- **Support engineers, operations teams, and managers:** Use System Design concepts to troubleshoot incidents, plan capacity, design runbooks, and align technical decisions with business priorities.

Ultimately, System Design strengthens collaboration across teams by giving everyone a shared understanding of how the system functions end-to-end.

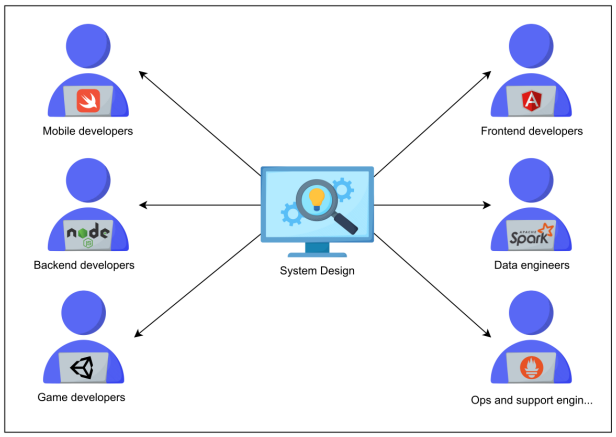


Figure 1.3: System Design enabling developers and engineers across roles

Understanding System Design gives developers a clearer perspective on how individual components fit into the larger system. This knowledge improves collaboration across teams and enables the creation of more reliable, scalable, and efficient applications, regardless of role.

2.3 Real-world applications that prove the need

Modern software doesn't operate in isolation. Whether serving millions or a smaller user base, real applications rely on distributed components that need careful coordination. Consider several practical scenarios where System Design knowledge is essential:

- **Rideshare applications:** Integrate multiple services such as messaging, payment processing, navigation, and user location tracking. Developers must coordinate these components to ensure reliability, performance, and scalability. System elements, such as load balancers, databases, and caching layers, are crucial for efficiently handling high traffic while maintaining responsiveness.
- **Cloud-based gaming:** Requires systems capable of supporting thousands of concurrent users with minimal latency. Multiplayer environments demand thoughtful design of real-time communication, data replication, and state management to deliver consistent and synchronized experiences.
- **Search engines:** Involve complex interactions among indexing, ranking, query handling, personalized recommendations, and ad delivery services. A strong grasp of System Design principles allows developers to optimize performance, manage distributed data, and minimize latency.
- **Dynamic web applications:** Depend on multiple backend systems, APIs, and databases to serve personalized and real-time content. Understanding design patterns and data flow ensures that user interactions are processed efficiently, securely, and reliably.

Note: Real-world systems underscore the importance of System Design, where every component, interaction, and potential failure has a significant impact. Applying core design principles ensures software remains reliable, scalable, and maintainable even under complex, real-world conditions.
These examples demonstrate that, regardless of the domain or scale, System Design knowledge is foundational to building software that performs well and stands the test of real-world demands.

2.4 What System Design covers in day-to-day engineering

System Design guides engineers in maintaining and improving systems reliably and efficiently. It connects everyday engineering tasks such as monitoring system health, responding to incidents, tuning performance, and iterating on features to broader architectural principles.

In practical terms, day-to-day System Design work includes:

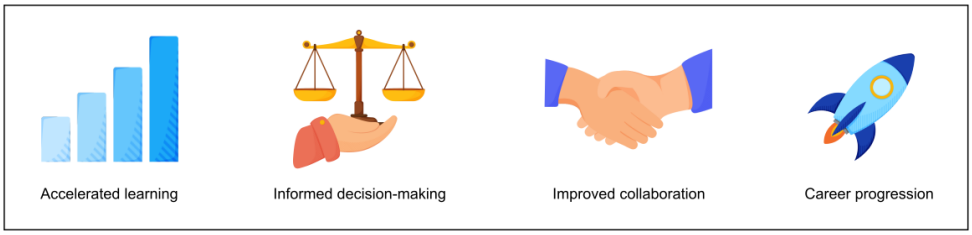
- **Monitoring and observability:** Tracking latency, error rates, throughput, and resource utilization to ensure the system meets its operational goals.
- **Incident response and troubleshooting:** Diagnosing failures, identifying root causes, and implementing fixes while minimizing user impact.
- **Capacity management:** Continuously evaluating traffic patterns, storage growth, and scaling requirements to prevent bottlenecks.
- **Performance tuning:** Adjusting caching strategies, load balancing, database queries, and other components to maintain efficiency under changing workloads.
- **System evolution and refactoring:** Iteratively improving services, updating dependencies, and introducing new components without disrupting existing functionality.
- **Operational automation:** Maintaining alerts, runbooks, structured logging, and automated health checks to reduce manual overhead and improve reliability.

Mastering these routine practices ensures systems operate smoothly, remain resilient under load, and evolve safely as requirements change.

2.5 Career benefits of learning System Design

Learning System Design early gives developers advantages that extend beyond day-to-day coding. The following benefits highlight how System Design shapes career growth, decision-making, and collaboration:

- **Accelerated learning:** Grasping system-level principles early in a career compounds knowledge, enabling faster problem-solving and deeper intuition for complex systems.
- **Informed decision-making:** Developers can evaluate trade-offs and anticipate issues such as latency, throughput, and failure modes before deployment.
- **Improved collaboration:** Shared understanding of system architecture enables better communication with peers, cross-functional teams, and managers.
- **Career progression:** Proficiency in System Design is often a prerequisite for senior engineering and leadership roles, providing a competitive advantage in technical and managerial career paths.



Early engagement strengthens problem-solving skills and equips developers to contribute effectively to complex projects.

2.6 A practical roadmap to learning System Design

Developing System Design expertise benefits from a clear, step-by-step approach. The following roadmap outlines practical areas and actions that help developers build intuition, tackle complex systems, and grow proficiency over time:

- **Review foundational concepts:** Networking protocols, consistency models, latency and throughput trade-offs, and fundamentals of distributed systems.

- **Data modeling:** Define schema and access patterns to guide storage decisions.
- **Core patterns:** Implement caching strategies, replication, sharding, leader/follower models, queues, and streams.
- **Estimate system requirements:** Calculate expected user load, storage, and network demands.
- **Visualize workflows:** Create sequence diagrams and request flows for critical paths.
- **Document designs:** Outline problems, requirements, alternatives, trade-offs, rollout plans, and risk mitigation strategies.
- **Iterate and test:** Begin with small systems, progressively tackling larger problems, and validate through monitoring and failure testing.
- **Operationalize:** Implement dashboards, SLIs/SLOs, runbooks, and simulated failure exercises to ensure system robustness.

System Design proficiency grows through deliberate practice and exposure to real-world scenarios.
Consistently applying design principles helps developers recognize patterns, anticipate challenges, and make informed decisions in complex systems. Repeated application of these steps develops intuition, competence, and the ability to design scalable systems effectively.
Note: A structured checklist for requirements, data modeling, failure scenarios, and monitoring helps maintain robust and production-ready systems.
Avoiding common pitfalls requires proactive thinking and foresight. Cultivating habits like anticipating edge cases, planning for failures, and embedding observability early strengthens systems and reduces operational risk.

2.7 Conclusion

System Design equips developers with the perspective required to operate effectively in modern software systems.
It enhances professional capabilities, improves decision-making, and enables the development of systems that are scalable, reliable, and maintainable. Early engagement with System Design principles enables developers to anticipate challenges, collaborate effectively, and contribute to the development of high-quality software systems.
Understanding why developers should learn System Design establishes the foundation for subsequent lessons, where specific design practices and real-world applications will be explored in detail.

3 Course Structure for Modern System Design

3.1 Structure of the course

This course consists of 40+ chapters. These chapters can be segmented into four distinct sections, as outlined below.

1. **Introduction:** The introduction section comprises five chapters. The first chapter provides an overview of what System Design is and why it's important for every developer. The second chapter introduces the System Design interview and explains the role of mock interviews in preparing for it. The third chapter covers preliminary System Design concepts. Next, we discuss key non-functional requirements that every large-scale system should have. Finally, we wrap up this section with a back-of-the-envelope calculation that helps estimate the resources required for design problems.

2. **Building blocks:** The "Building Blocks" chapter begins with an introductory lesson that presents sixteen different building blocks. Each of these building blocks is explained in an independent chapter. We conclude this section with the "Conclusion", which also serves as an introduction to the next section.

3. **Design problems:** This section is the meat of the course and is carefully crafted from sixteen design problems.

4. **Epilogue:** The "Epilogue" section wraps up this course and is divided into three parts. The first part, spectacular failures, explains how, in the real world, even a small mistake can bring down a large and successful system. Although such failures may be inevitable, we highlight several measures to help mitigate them. The second part provides concluding remarks that summarize what we have learned in the course, and the final part includes additional chapters that offer supplementary resources for further strengthening your System Design concepts.

   Note: Although we did our best to keep the chapters independent, our readers will find it useful to read them in the given sequence.

The structure of this course

[Interactive Mind Map - See online version for interactive visualization]

- **This course**
  - **Introduction**
    - Introduction
    - System Design Interviews
    - Prelimenary System Design Concepts
    - Non-functional system characteristics
    - Back-of-the-envelope calculations
  - **Building Blocks**
    - Introduction
    - DNS
    - Load balancers
    - Databases
    - Key-value store
    - CDN
    - Sequencer
    - Distributed monitoring
    - Distributed cache
    - Distributed messaging queue
    - Pub-sub system
    - Rate limiter
    - Blob store
    - Distributed search
    - Distributed logging
    - Distributed task scheduler
    - Sharded counters
    - Concluding building blocks

- **Design Problems**
  - Design YouTube
  - Design Quora
  - Design Google Maps
  - Design Yelp
  - Design Uber
  - Design Twitter
  - Design Newsfeed system
  - Design Instagram
  - Design TinyURL
  - Design Web crawler
  - Design WhatsApp
  - Design Typeahead
  - Design Google Docs
  - Design Deployment system
  - Design Payment system
  - Design ChatGPT
- **Epilogue**
  - Spectacular failures
  - Concluding remarks
  - Additional resources

3.2 Strengths of the course

While filling some important gaps in other available courses, we believe this course has some key strengths to offer. We summarize the strengths and the advantages this course has over others in the table given below.

Strengths	Advantage
Building blocks	This is a modern approach to system design where we construct bigger artifacts using smaller building blocks.
Building blocks as design problems	We'll treat each one of our building blocks as a stand-alone, mini design problem.
Incremental improvement to design	Layer-by-layer design solution addresses added bottlenecks, designing simple and incremental solutions to complex systems.
Evaluating the design	Accountability of the provided design solution shows the performance of our design.
Solving the traditional problems with updated designs	This course is up to date with the latest industry demands.
New design problems added	This course contains updates to decades-old system design courses.
Careful collection of design problems	Each problem has its unique aspects in terms of problem-solving and designing.
Contributions by experts from FAANG	Learn from the best.
AI-based quizzes and assessments	These assessments and interviews evaluate learner's concepts of system design.

Let's start our system design journey!